

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
IMPLEMENTASI EXCEPTION DAN ERROR HANDLING
PADA SISTEM RESERVASI TIKET
"JAVA EXPRESS"



Disusun Oleh:

Glory Hardy Febriando Sianturi

(105223013)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Program "JAVA EXPRESS" adalah sebuah sistem simulasi reservasi tiket kereta api interaktif berbasis *Command Line Interface* (CLI). Tujuan utama dari program ini adalah mengimplementasikan arsitektur penanganan kesalahan (*Exception* dan *Error Handling*) di dalam Java secara komprehensif untuk memastikan stabilitas dan keandalan aplikasi.

Tantangan utama dalam pengembangan sistem ini adalah merancang perlindungan kode berlapis untuk mencegah aplikasi berhenti mendadak (*crash*) atau terjebak dalam perulangan tanpa henti (*infinite loop*) akibat kesalahan *input* dari pengguna. Selain itu, sistem ini menerapkan aturan bisnis yang ketat melalui pembuatan *Custom Exception* (*Checked* dan *Unchecked Exception*) untuk memvalidasi format Nomor Induk Kependudukan (NIK) penumpang, memverifikasi ketersediaan rute kereta, dan mengontrol kapasitas sisa kursi secara aman.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	Kode	String	Menyimpan identitas unik kereta api sebagai kunci pencarian.
2	nama	String	Menyimpan deskripsi nama armada kereta api.
3	rute	String	Menyimpan informasi rute perjalanan kereta.
4	sisaKursi	int	Menyimpan jumlah ketersediaan kursi yang bisa dipesan..
5	daftarKereta	List	Menyimpan objek kereta awal di dalam memori saat sistem diinisialisasi.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KeretaApi()	<i>Constructor</i>	Menginisialisasi atribut dasar (kode, nama, rute, sisaKursi) saat objek dicetak.
2	ReservasiController()	<i>Constructor</i>	Menginisialisasi <i>list</i> data awal kereta otomatis ke dalam memori aplikasi.
3	tambahStok()	<i>Procedural</i>	Melakukan iterasi untuk mencetak jadwal, rute, dan sisa kursi ke layar terminal.
4	pesanTiket()	<i>Procedural</i>	Mengeksekusi logika pemesanan dengan validasi <i>Exception</i> secara berlapis.
5	kurangiKursi()	<i>Procedural</i>	Mengurangi nilai atribut ketersediaan kursi berdasarkan jumlah tiket yang dipesan.

IV. Dokumentasi dan Pembahasan Code

4.1. Pembuatan Class Model (KeretaApi.java)

Pada bagian ini, sistem mendefinisikan entitas murni (*Plain Old Java Object*) yang bertugas menampung data kereta api beserta metode untuk memanipulasi kapasitas kursinya.

Kode KeretaApi.java :

```
Windsurf: Refactor | Explain
1 public class KeretaApi {
2     private String kode;
3     private String nama;
4     private String rute;
5     private int sisaKursi;
6
7     public KeretaApi(String kode, String nama, String rute, int sisaKursi) {
8         this.kode = kode;
9         this.nama = nama;
10        this.rute = rute;
11        this.sisaKursi = sisaKursi;
12    }
13
14    public String getKode() { return kode; }
15    public String getNama() { return nama; }
16    public String getRute() { return rute; }
17    public int getSisaKursi() { return sisaKursi; }
18
19    Windsurf: Refactor | Explain | Generate Javadoc | X
20    public void kurangiKursi(int jumlah) {
21        this.sisaKursi -= jumlah;
22    }
23 }
```

Pembahasan:

Kelas ini difokuskan pada enkapsulasi data. Atribut dibuat private dan diakses melalui *getter*, sementara metode *kurangiKursi()* bertugas memastikan operasi pengurangan sisa kursi berjalan terisolasi di dalam objek yang bersangkutan.

4.2. Implementasi Custom Exception

Sistem mewajibkan perlindungan aturan bisnis menggunakan kelas *Exception* buatan sendiri agar pesan *error* lebih spesifik dan arsitektur lebih rapi.

Kode DataPenumpangTidakValidException.java:

```
Windsurf: Refactor | Explain
1 public class DataPenumpangTidakValidException extends RuntimeException {
2     public DataPenumpangTidakValidException(String message) {
3         super(message);
4     }
5 }
```

Kode RuteTidakDitemukanException

```
1 public class RuteTidakDitemukanException extends Exception {  
2     public RuteTidakDitemukanException(String message) {  
3         super(message);  
4     }  
5 }
```

Kode TiketHabisException.java

```
Windsurf: Refactor | Explain  
1 public class TiketHabisException extends Exception {  
2     public TiketHabisException(String namaKereta, int sisaKursi) {  
3         super("Tiket habis! Kereta " + namaKereta + " hanya tersisa " + sisaKursi + " kursi.");  
4     }  
5 }
```

Pembahasan:

- DataPenumpangTidakValidException diturunkan dari RuntimeException (*Unchecked Exception*) karena kesalahan format NIK (bukan 16 digit angka) adalah murni kesalahan *input* yang harus digagalkan langsung tanpa paksaan pengecekan waktu kompilasi.
- RuteTidakDitemukanException dan TiketHabisException diturunkan dari Exception (*Checked Exception*) untuk memaksa *programmer* secara eksplisit menangani potensi kegagalan transaksi ini di dalam blok *try-catch* saat pemesanan terjadi.

4.3. Pusat Kontrol Reservasi (ReservasiController.java)

Kelas ini bertindak sebagai pengendali logika utama yang menjembatani data model dengan interaksi pengguna, serta mendeklarasikan potensi *error*.

Kode ReservasiController.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 Windsurf: Refactor | Explain
5 public class ReservasiController {
6     private List<KeretaApi> daftarKereta;
7
8     public ReservasiController() {
9         daftarKereta = new ArrayList<>();
10        daftarKereta.add(new KeretaApi(kode: "K01", nama: "Argo Bromo", rute: "JKT - SBY", sisaKursi: 50));
11        daftarKereta.add(new KeretaApi(kode: "K02", nama: "Parahyangan", rute: "JKT - BDG", sisaKursi: 15));
12    }
13
14 Windsurf: Refactor | Explain | Generate Javadoc | X
15 public void tampilkanJadwal() {
16     System.out.println(x: "\n=== JADWAL KERETA API JAVA EXPRESS ===");
17     for (KeretaApi k : daftarKereta) {
18         System.out.println(k.getKode() + " | " + k.getNama() + " (" + k.getRute() + ") - Sisa Kursi: " + k.getSisaKursi());
19     }
20     System.out.println(x: "=====");
21 }
22
23 Windsurf: Refactor | Explain | Generate Javadoc | X
24 public void pesanTiket(String kode, String nik, String namaPenumpang, int jumlahTiket)
25     throws RuteTidakDitemukanException, TiketHabisException {
26
27     if (nik == null || nik.length() != 16 || !nik.matches(regex: "\\d+")) {
28         throw new DataPenumpangTidakValidException(message: "Format NIK salah. NIK harus terdiri dari 16 digit angka.");
29     }
30
31     KeretaApi keretaPilihan = null;
32     for (KeretaApi k : daftarKereta) {
33         if (k.getKode().equalsIgnoreCase(kode)) {
34             keretaPilihan = k;
35             break;
36         }
37     }
38
39     if (keretaPilihan == null) {
40         throw new RuteTidakDitemukanException("Kode kereta '" + kode + "' tidak terdaftar di sistem.");
41     }
42
43     if (jumlahTiket > keretaPilihan.getSisaKursi()) {
44         throw new TiketHabisException(keretaPilihan.getNama(), keretaPilihan.getSisaKursi());
45     }
46
47     keretaPilihan.kurangiKursi(jumlahTiket);
48     System.out.println(x: "\n✅ RESERVASI BERHASIL!");
49     System.out.println(x: "Detail Pemesanan:");
50     System.out.println("- Penumpang : " + namaPenumpang + " (NIK: " + nik + ")");
51     System.out.println("- Kereta : " + keretaPilihan.getNama() + " (" + keretaPilihan.getRute() + ")");
52     System.out.println("- Jumlah Tiket : " + jumlahTiket);
53 }

```

Pembahasan:

Metode pesanTiket() secara eksplisit menggunakan kata kunci throws untuk mendeklarasikan *Checked Exception*. Pengecekan dilakukan secara berurutan: NIK divalidasi dengan *Regex* `\\d+`, pencarian kereta dilakukan dengan membandingkan teks, dan terakhir kapasitas kursi diverifikasi sebelum stok dikurangi.

4.4 Simulasi Eksekusi & Error Handling (JavaExpressApp.java)

Kelas ini bertindak sebagai pengendali logika utama yang menjembatani data model dengan interaksi pengguna, serta mendeklarasikan potensi *error*.

Kode JavaExpressApp.java

```

1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3
4 public class JavaExpressApp {
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7         ReservasiController controller = new ReservasiController();
8         boolean isRunning = true;
9
10        System.out.println("Selamat datang di Sistem Reservasi Tiket JAVA EXPRESS");
11
12        while (isRunning) {
13            try {
14                System.out.println("\n--- MENU UTAMA ---");
15                System.out.println("1. Lihat Jadwal");
16                System.out.println("2. Pesan Tiket");
17                System.out.println("3. Keluar");
18                System.out.print("Pilih menu (1-3): ");
19
20                int pilihan = scanner.nextInt();
21                scanner.nextLine();
22
23                switch (pilihan) {
24                    case 1:
25                        controller.tampilkanJadwal();
26                        break;
27                    case 2:
28                        System.out.print("Masukkan Kode Kereta: ");
29                        String kode = scanner.nextLine();
30                        System.out.print("Masukkan NIK Penumpang: ");
31                        String nik = scanner.nextLine();
32                        System.out.print("Masukkan Nama Penumpang: ");
33                        String nama = scanner.nextLine();
34                        System.out.print("Masukkan Jumlah Tiket: ");
35
36                        int jumlah = scanner.nextInt();
37                        scanner.nextLine();
38
39                        controller.pesanTiket(kode, nik, nama, jumlah);
40                        break;
41                    case 3:
42                        isRunning = false;
43                        break;
44                    default:
45                        System.out.println("⚠ Pilihan tidak valid. Silakan ketik angka 1, 2, atau 3.");
46                }
47            } catch (InputMismatchException e) {
48                System.out.println("⚠ Error Input: Anda memasukkan format yang salah. Harap hanya memasukkan angka untuk Pilihan Menu dan Jumlah Tiket.");
49                scanner.nextLine();
50            } catch (DataPenumpangTidakValidException e) {
51                System.out.println("❌ Reservasi Gagal: " + e.getMessage());
52            } catch (RuteTidakDitemukanException | TiketHabisException e) {
53                System.out.println("❌ Reservasi Gagal: " + e.getMessage());
54            } finally {
55                if (!isRunning) {
56                    System.out.println("\n🔴 Sistem Ditutup. Terima kasih telah menggunakan JAVA EXPRESS!");
57                    scanner.close();
58                }
59            }
60        }
61    }
62 }

```

Pembahasan:

- Stabilitas Input: Jika pengguna memasukkan huruf pada menu angka, `InputMismatchException` bawaan Java akan ditangkap. Baris `scanner.nextLine()` dipanggil di dalam blok `catch` untuk membersihkan *buffer input* yang rusak, sehingga program terhindar dari *infinite loop*.
- Multiple Catch: Program menangkap berbagai jenis *exception* secara terpisah untuk memberikan pesan umpan balik (*feedback*) yang spesifik dan ramah kepada pengguna.
- Blok Finally: Saat sistem ditutup (`isRunning = false`), blok `finally` dipastikan tereksekusi untuk menutup *resource* memori `scanner.close()`.

V. Kesimpulan

Berdasarkan hasil perancangan arsitektur Sistem Reservasi Tiket "JAVA EXPRESS", dapat ditarik kesimpulan sebagai berikut:

1. Validasi Data yang Presisi: Penggunaan *Custom Unchecked Exception* sangat efektif dalam mencegah masuknya data kotor (seperti NIK berisi huruf) ke dalam alur pemrosesan sistem.

2. Stabilitas Aplikasi Terjamin: Penangkapan `InputMismatchException` yang dikombinasikan dengan pembersihan memori *buffer* (`scanner.nextLine()`) terbukti ampuh mencegah aplikasi *crash* atau *error* berkelanjutan saat menerima interaksi pengguna yang tidak terduga.
3. Keamanan Eksekusi: Penempatan instruksi penutupan alat masukan di dalam blok `finally` menjamin tidak adanya kebocoran memori (*memory leak*), karena blok tersebut mutlak dieksekusi terlepas dari apakah aplikasi berjalan lancar atau sempat mengalami *error*.

VI. Daftar Pustaka

- Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.
- Oracle. (2026). *Java SE Documentation: Exceptions*. [Online]. Tersedia di: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- Schildt, H. (2018). *Java: The Complete Reference*. McGraw-Hill Education.